I am working on a Pygame target shooting game. Here is the code for game/hit_detection.py and game/target.py. Currently, there is a bug where the hit detection uses pygame.Rect(self.x, self.y, radius*2, radius*2) but x, y is the center of the circle, offsetting the hitbox. Please fix check_hit in hit_detection.py so it calculates distance using the Pythagorean theorem (or math.hypot) to check if the click coordinates fall within the target's actual circular radius. Provide the updated files.
—x—x—x
hit-detection.py

```
"""
hit_detection: figures out whether a click landed on a target.
"""



def check_hit(targets, click_pos):
    """
    Returns the target that was clicked, or None if the click
missed
    every target.
    """
    for target in targets:
        rect = target.get_bounding_rect()
        if rect.collidepoint(click_pos):
            return target
    return None
```

target.py

```
"""
Target: a circular target the player clicks on. (x, y) is the
CENTER
of the circle - this matters for how it's drawn vs. how it's hit-
tested.
"""


import pygame
```

```
class Target:
    def __init__(self, x, y, radius=28, color=(230, 90, 70)):
        self.x = x
        self.y = y
        self.radius = radius
        self.color = color

    def get_bounding_rect(self):
        """A square bounding box around the circle - NOT the
same
        shape as the actual circle, and easy to build incorrectly."""
        return pygame.Rect(self.x, self.y, self.radius * 2,
self.radius * 2)
```

Here are the updated files. The hit detection now uses `math.hypot` to calculate the exact distance between the click and the target's center, ensuring accurate circular collision. I also corrected the offset issue in `get_bounding_rect` in `target.py` in case you need it for rendering or broad-phase collision checks later.

`hit_detection.py`

Python

```python
"""
hit_detection: figures out whether a click landed on a target.
"""
import math


def check_hit(targets, click_pos):
    """
    Returns the target that was clicked, or None if the click missed
    every target.
    """
    click_x, click_y = click_pos

    for target in targets:
```

```
        # Calculate the distance between click and the target's center
        distance = math.hypot(target.x - click_x, target.y - click_y)

        # If the distance is less than or equal to the radius, it's a hit
        if distance <= target.radius:
            return target

    return None
```

target.py

Python

```
"""
Target: a circular target the player clicks on. (x, y) is the CENTER
of the circle - this matters for how it's drawn vs. how it's hit-tested.
"""

import pygame


class Target:
    def __init__(self, x, y, radius=28, color=(230, 90, 70)):
        self.x = x
        self.y = y
        self.radius = radius
        self.color = color

    def get_bounding_rect(self):
        """A square bounding box around the circle."""
        # Offset the top-left corner by the radius so (x,y) remains the center
        return pygame.Rect(
            self.x - self.radius,
            self.y - self.radius,
            self.radius * 2,
            self.radius * 2
        )
```

Task 2: Now I need to make the targets move. Here is the updated game/target.py and game/game_engine.py. Please modify the Target class so targets move continuously within the play area and bounce off the screen edges. In

Here are the updated files. I added random velocities to the `Target` class with two distinct speed tiers (slow and fast) to vary the movement patterns, along with boundary collision logic so they bounce off the screen edges.

The `GameEngine`'s `update` method now iterates through the active targets and advances their positions every frame. Because we update the target's internal `x` and `y` properties, the `check_hit` function from Task 1 will automatically test against their actual live positions.

`target.py`

Python

```python
"""
Target: a circular target the player clicks on. (x, y) is the CENTER
of the circle - this matters for how it's drawn vs. how it's hit-tested.
"""

import pygame
import random


class Target:
    def __init__(self, x, y, radius=28, color=(230, 90, 70)):
        self.x = x
        self.y = y
        self.radius = radius
        self.color = color

        # Assign random x/y velocities with at least two different speed tiers
        speed_tier = random.choice([2, 6])  # 2 = slow target, 6 = fast target
```

```
        # Multiply by a random direction (-1 or 1) and add a slight random angle
        self.vx = random.choice([-1, 1]) * random.uniform(0.7, 1.3) * speed_tier
        self.vy = random.choice([-1, 1]) * random.uniform(0.7, 1.3) * speed_tier

    def move(self, screen_width, screen_height):
        """Advances the target's position and handles edge bouncing."""
        self.x += self.vx
        self.y += self.vy

        # Bounce off left/right edges
        if self.x - self.radius <= 0:
            self.x = self.radius
            self.vx *= -1
        elif self.x + self.radius >= screen_width:
            self.x = screen_width - self.radius
            self.vx *= -1

        # Bounce off top/bottom edges
        if self.y - self.radius <= 0:
            self.y = self.radius
            self.vy *= -1
        elif self.y + self.radius >= screen_height:
            self.y = screen_height - self.radius
            self.vy *= -1

    def get_bounding_rect(self):
        """A square bounding box around the circle."""
        return pygame.Rect(
            self.x - self.radius,
            self.y - self.radius,
            self.radius * 2,
            self.radius * 2
        )
```

game_engine.py

Python

```
"""
GameEngine: owns the targets and handles player clicks.

Starter version: a few static targets that respawn elsewhere when
```

```
Starter version: a few static targets that respawn elsewhere when
clicked (correctly or not, depending on the hit-detection bug) - no
movement, no score/combo, no timer yet. That's Tasks 2-4.
"""

import random

from game.target import Target
from game.hit_detection import check_hit
from game.renderer import WIDTH, HEIGHT

NUM_TARGETS = 3
TARGET_RADIUS = 28


class GameEngine:
    def __init__(self):
        self.targets = [self._random_target() for _ in range(NUM_TARGETS)]
        self.hits = 0
        self.misses = 0

    def _random_target(self):
        x = random.randint(TARGET_RADIUS + 10, WIDTH - TARGET_RADIUS - 10)
        y = random.randint(TARGET_RADIUS + 10, HEIGHT - TARGET_RADIUS - 10)
        return Target(x, y, radius=TARGET_RADIUS)

    def handle_click(self, pos):
        target = check_hit(self.targets, pos)
        if target is not None:
            self.hits += 1
            self.targets.remove(target)
            self.targets.append(self._random_target())
        else:
            self.misses += 1

    def update(self):
        """Updates game state every frame."""
        for target in self.targets:
            target.move(WIDTH, HEIGHT)

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.targets)
        renderer.draw_text(surface, font, f"Hits: {self.hits}  Misses: {self.mis
```

Task 3: Implement combo scoring. Here are the current game files (game_engine.py, renderer.py). I need to add a score and a combo multiplier. Each hit should add points based on the current combo multiplier. Consecutive hits (nc

Here are the updated files. I added `score` and `combo` variables to the `GameEngine`, implemented the point logic using a base score, and added a dedicated `draw_hud` function in `renderer.py` to cleanly display the new statistics on screen.

`game_engine.py`

Python

```python
"""
GameEngine: owns the targets and handles player clicks.
"""

import random

from game.target import Target
from game.hit_detection import check_hit
from game.renderer import WIDTH, HEIGHT

NUM_TARGETS = 3
TARGET_RADIUS = 28
BASE_POINTS = 100


class GameEngine:
    def __init__(self):
        self.targets = [self._random_target() for _ in range(NUM_TARGETS)]
        self.hits = 0
        self.misses = 0
        self.score = 0
        self.combo = 1
```

```
        self.combo = 1

    def _random_target(self):
        x = random.randint(TARGET_RADIUS + 10, WIDTH - TARGET_RADIUS - 10)
        y = random.randint(TARGET_RADIUS + 10, HEIGHT - TARGET_RADIUS - 10)
        return Target(x, y, radius=TARGET_RADIUS)

    def handle_click(self, pos):
        target = check_hit(self.targets, pos)
        if target is not None:
            # Register hit: Add score based on combo, increment stats
            self.hits += 1
            self.score += BASE_POINTS * self.combo
            self.combo += 1  # Increase multiplier for the next consecutive hit

            self.targets.remove(target)
            self.targets.append(self._random_target())
        else:
            # Register miss: Reset combo multiplier
            self.misses += 1
            self.combo = 1

    def update(self):
        """Updates game state every frame."""
        for target in self.targets:
            target.move(WIDTH, HEIGHT)

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.targets)
        renderer.draw_hud(surface, font, self.score, self.combo, self.hits, self
```

renderer.py

Python

```
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame
```

```
WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (25, 25, 35)
COLOR_TEXT = (255, 255, 255)
COLOR_COMBO = (255, 200, 50)  # Highlight color for active combos


def draw_scene(surface, targets):
    surface.fill(COLOR_BG)
    for target in targets:
        pygame.draw.circle(surface, target.color, (int(target.x), int(target.y))
        pygame.draw.circle(surface, (255, 255, 255), (int(target.x), int(target.


def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_hud(surface, font, score, combo, hits, misses):
    """Draws the Heads Up Display with score, combo, and stats."""
    # Main stats row
    stats_text = f"Score: {score}    Hits: {hits}    Misses: {misses}"
    draw_text(surface, font, stats_text, (10, 10))

    # Combo multiplier (drawn below the main stats)
    combo_text = f"Combo: {combo}x"
    # Highlight the combo text in gold if the player has a streak going
    color = COLOR_COMBO if combo > 1 else COLOR_TEXT
    draw_text(surface, font, combo_text, (10, 40), color=color)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
    rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height()
    surface.blit(surf, rect)
```

Task 4: Implement a 30-second timed round. Here are the current files. Please add a countdown timer that starts at 30

seconds. Display the remaining time on the screen. When the timer reaches zero, the game state should change

Here are the updated files.

I restructured `GameEngine` to use a `state` variable and a `reset()` method so it can easily transition from `'PLAYING'` to `'GAME_OVER'` and back. The timer uses Pygame's built-in tick counter to calculate delta time ( `dt` ) independently of frame rate.

In the renderer, I added a dark, semi-transparent overlay to make the Game Over screen and final score pop, and placed the countdown timer in the top right corner.

`game_engine.py`

**Python**

```python
"""
GameEngine: owns the targets and handles player clicks, scoring, and game state.
"""

import random
import pygame

from game.target import Target
from game.hit_detection import check_hit
from game.renderer import WIDTH, HEIGHT

NUM_TARGETS = 3
TARGET_RADIUS = 28
BASE_POINTS = 100
ROUND_TIME = 30.0  # 30-second rounds


class GameEngine:
    def __init__(self):
        self.reset()

    def reset(self):
        """Resets all game variables for a fresh round."""
        self.targets = [self._random_target() for _ in range(NUM_TARGETS)]
        self.hits = 0
```

```
        self.hits = 0
        self.misses = 0
        self.score = 0
        self.combo = 1

        self.state = "PLAYING"
        self.time_left = ROUND_TIME
        self.last_time = pygame.time.get_ticks()

    def _random_target(self):
        x = random.randint(TARGET_RADIUS + 10, WIDTH - TARGET_RADIUS - 10)
        y = random.randint(TARGET_RADIUS + 10, HEIGHT - TARGET_RADIUS - 10)
        return Target(x, y, radius=TARGET_RADIUS)

    def handle_click(self, pos):
        if self.state == "GAME_OVER":
            # Any click restarts the game
            self.reset()
            return

        target = check_hit(self.targets, pos)
        if target is not None:
            self.hits += 1
            self.score += BASE_POINTS * self.combo
            self.combo += 1

            self.targets.remove(target)
            self.targets.append(self._random_target())
        else:
            self.misses += 1
            self.combo = 1

    def handle_keydown(self, key):
        """Allows spacebar restarts if hooked up to the main event loop."""
        if self.state == "GAME_OVER" and key == pygame.K_SPACE:
            self.reset()

    def update(self):
        """Updates game state every frame."""
        current_time = pygame.time.get_ticks()
        dt = (current_time - self.last_time) / 1000.0
        self.last_time = current_time

        if self.state == "GAME_OVER":
            return
```

```python
        # Update timer
        self.time_left -= dt
        if self.time_left <= 0:
            self.time_left = 0
            self.state = "GAME_OVER"

        # Move targets
        for target in self.targets:
            target.move(WIDTH, HEIGHT)

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.targets)
        renderer.draw_hud(surface, font, self.score, self.combo, self.hits, self

        if self.state == "GAME_OVER":
            renderer.draw_game_over(surface, font, self.score)
```

renderer.py

Python

```python
"""
renderer: all pygame drawing lives here, kept separate from game logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (25, 25, 35)
COLOR_TEXT = (255, 255, 255)
COLOR_COMBO = (255, 215, 0)
COLOR_TIME_LOW = (255, 100, 100)


def draw_scene(surface, targets):
    surface.fill(COLOR_BG)
    for target in targets:
        pygame.draw.circle(surface, target.color, (int(target.x), int(target.y))
        pygame.draw.circle(surface, (255, 255, 255), (int(target.x), int(target.
```

```python
def draw_text(surface, font, text, pos, color=COLOR_TEXT, center=False):
    """Helper to draw text, optionally centering it on the given position."""
    surf = font.render(text, True, color)
    if center:
        rect = surf.get_rect(center=pos)
        surface.blit(surf, rect)
    else:
        surface.blit(surf, pos)


def draw_hud(surface, font, score, combo, hits, misses, time_left):
    """Draws all on-screen statistics."""
    # Draw Score & Combo
    draw_text(surface, font, f"Score: {score}", (10, 10))

    combo_color = COLOR_COMBO if combo > 1 else COLOR_TEXT
    draw_text(surface, font, f"Combo: {combo}x", (10, 40), color=combo_color)

    # Draw Hits/Misses
    draw_text(surface, font, f"Hits: {hits}  Misses: {misses}", (10, HEIGHT - 35

    # Draw Timer (Right-aligned, turns red at 5 seconds)
    time_color = COLOR_TIME_LOW if time_left <= 5 else COLOR_TEXT
    time_str = f"Time: {max(0, int(time_left))}s"
    time_surf = font.render(time_str, True, time_color)
    surface.blit(time_surf, (WIDTH - time_surf.get_width() - 15, 10))


def draw_game_over(surface, font, final_score):
    """Draws the game over overlay and final score."""
    # Dark semi-transparent background overlay
    overlay = pygame.Surface((WIDTH, HEIGHT))
    overlay.set_alpha(190)
    overlay.fill((0, 0, 0))
    surface.blit(overlay, (0, 0))

    # Game Over Text
    draw_text(surface, font, "TIME'S UP!", (WIDTH // 2, HEIGHT // 2 - 40), color
    draw_text(surface, font, f"Final Score: {final_score}", (WIDTH // 2, HEIGHT
    draw_text(surface, font, "Click anywhere or press SPACE to restart", (WIDTH


def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
```

```
surf = font.render(text, True, (255, 220, 80))
rect = surf.get_rect(center=(surface.get_width() // 2, surface.get_height()
surface.blit(surf, rect)
```